ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DESENVOLVIMENTO WEB

GRUPO 1

React no desenvolvimento web

Roteiro de divisão do trabalho entre cinco integrantes

Organização escrita das responsabilidades, na sequência da equipe, com explicação do conteúdo e das entregas de cada participante.

Mauro César Maia Reis
Matrícula 04214707

Eduardo Gabriel Soares Ferreira
Matrícula 04209877

Patrick de Oliveira Almeida
Matrícula 26075797

Caio Renato Nascimento Donati
Matrícula 04214983

Luís Mário Marques Costa
Matrícula 04214724

Este roteiro complementa o relatório acadêmico. Não prevê apresentação oral nem slides. A divisão mantém as responsabilidades propostas no Word e distribui os tópicos de pesquisa para revisão conjunta.

Registro de participação: as atribuições abaixo devem ser confirmadas pela equipe. Código, testes, instalação e publicação ainda dependem de execução; não são apresentados como atividades concluídas.

1 Primeiro integrante

Mauro César Maia Reis

Matrícula 04214707

Introdução, contexto histórico e fundamentos do React

Parte atribuída no trabalho

Mauro fica responsável por organizar a abertura da pesquisa, apresentar seu objetivo e explicar a base teórica. Essa parte corresponde às seções 1, 2 e 3 do relatório: introdução, contextualização histórica e fundamentação teórica. Também coordena a revisão da coerência do texto completo, sem substituir as responsabilidades dos demais.

Roteiro explicado

Começar pelo problema das interfaces web: telas precisam acompanhar mudanças nos dados sem perder consistência. Em seguida, apresentar React como biblioteca JavaScript de interfaces, esclarecendo por que o enunciado o agrupa com frameworks, embora seu núcleo tenha uma responsabilidade mais delimitada. Relacionar essa diferença às escolhas complementares de um projeto.

Depois, contextualizar sua origem no Facebook, hoje Meta, e a abertura pública em 2013. Explicar a motivação de organizar interfaces dinâmicas. Destacar Hooks em React 16.8 e os recursos de React 18, evitando transformar a história em uma lista de versões sem relação com os problemas resolvidos.

Concluir sua parte explicando desenvolvimento declarativo, componentes, props e JSX. Usar como exemplo a lista de tarefas: a interface decorre dos dados, e componentes menores representam formulário, lista e indicadores. Diferenciar entrada recebida por props de informação mantida em estado.

O que deverá entregar e revisar

Texto de introdução e fundamentos com as fontes indicadas; conferência dos conceitos e da terminologia; verificação de que objetivos, método e conclusão do relatório sejam compatíveis. Na aplicação, apoiar a definição do objetivo do Organiza Estudos e do limite de funcionalidades.

Ligação com o próximo integrante

A base conceitual preparada por Mauro permite que Eduardo explique como esses conceitos são organizados na árvore de componentes e nas partes visuais da aplicação.

Síntese da participação

Mauro: pesquisa de origem e fundamentos, organização da introdução e coordenação da coerência acadêmica.

2 Segundo integrante

Eduardo Gabriel Soares Ferreira

Matrícula 04209877

Arquitetura, componentes e organização da interface

Parte atribuída no trabalho

Eduardo fica responsável por detalhar a arquitetura da interface e a decomposição em componentes. Sua pesquisa se concentra na seção 4, na componentização da seção 5 e nas ferramentas de desenvolvimento da seção 6. Na proposta prática da seção 11, organiza a apresentação visual e os limites dos componentes.

Roteiro explicado

Iniciar mostrando que a aplicação forma uma árvore: App coordena a tela, enquanto componentes menores representam tarefas específicas. Explicar que renderização é o cálculo da descrição da interface; commit é a aplicação das mudanças necessárias ao DOM. Uma renderização não equivale a reconstruir toda a página.

Em seguida, apresentar os componentes propostos. TaskForm organiza o preenchimento; TaskList recebe os itens filtrados; TaskItem mostra um registro e suas ações; TaskFilters apresenta os critérios de consulta; TaskSummary exibe as contagens. Explicar quais dados cada um recebe e quais solicitações devolve por funções.

Relacionar reutilização a responsabilidades claras. Um campo com rótulo e mensagem de erro pode ser reaproveitado, mas dividir cada elemento em um componente sem finalidade aumenta a complexidade. Explicar também por que cada tarefa precisa de identificador estável para evitar associações incorretas em listas.

O que deverá entregar e revisar

Organização dos componentes e da interface, com nomes compreensíveis e comportamento consistente. A disposição visual deverá permitir leitura em telas pequenas, uso por teclado e identificação dos campos por rótulos. A revisão deve observar mensagens de erro, lista vazia, foco e clareza das ações.

Documentar o papel do Vite no ambiente de desenvolvimento e do React Developer Tools na inspeção de componentes, props e estado. Essas ferramentas apoiam a investigação; sua presença não comprova, por si, desempenho ou qualidade.

Ligação com o próximo integrante

Eduardo define a estrutura visual. Patrick explica como os dados circulam nessa estrutura e como os eventos atualizam estado, filtros e indicadores.

Síntese da participação

Eduardo: arquitetura de componentes, apresentação visual, reutilização e apoio das ferramentas de desenvolvimento.

3 Terceiro integrante

Patrick de Oliveira Almeida

Matrícula 26075797

Estado, eventos, filtros e avaliação dos recursos

Parte atribuída no trabalho

Patrick fica responsável pelo gerenciamento de estado e pelo funcionamento dos filtros. Essa parte corresponde aos recursos da seção 5, à discussão de vantagens e limitações da seção 8 e às regras de interação da seção 11. A implementação deverá demonstrar como alterações nos dados produzem mudanças na interface.

Roteiro explicado

Começar identificando a fonte de verdade: a coleção principal de tarefas. O texto em edição pode ficar no formulário, enquanto a coleção consultada por lista e indicadores deve ficar no componente responsável por compartilhar esses dados. Explicar elevação de estado e o motivo de evitar cópias independentes da mesma informação.

Depois, apresentar useState para mudanças simples e useReducer como possibilidade quando as ações ficarem mais complexas. Explicar que Context facilita acesso a dados distantes na árvore, mas não deve concentrar indiscriminadamente toda a aplicação. A escolha precisa responder a uma necessidade real.

Detalhar o fluxo das ações: cadastrar, editar, concluir e excluir. Atualizações devem preservar o identificador da tarefa e produzir estruturas coerentes. O total de concluídas e a lista filtrada devem ser calculados a partir da coleção, evitando guardar resultados derivados como estados separados sem necessidade.

Na filtragem, combinar texto e situação de maneira previsível. Uma tarefa concluída deverá deixar de aparecer quando o filtro estiver restrito às pendentes. Explicar o resultado esperado e verificar que os indicadores não entrem em contradição com os registros.

O que deverá entregar e revisar

Regras de estado e filtros, documentação das ações e exemplos de atualização. Na pesquisa, discutir pelo menos cinco vantagens, como reutilização, declaração de estados visuais, fluxo rastreável, flexibilidade e separação de lógica; e três desafios, incluindo decisões externas ao núcleo, erros de estado e necessidade de medir desempenho.

Ligação com o próximo integrante

Com o fluxo de dados definido por Patrick, Caio organiza sua persistência e os testes necessários para verificar os comportamentos previstos.

Síntese da participação

Patrick: estado, eventos, filtros, indicadores e análise crítica dos benefícios e desafios dos recursos.

4 Quarto integrante

Caio Renato Nascimento Donati

Matrícula 04214983

Persistência, integração com APIs e testes

Parte atribuída no trabalho

Caio fica responsável pela persistência e pelo planejamento da validação. Sua pesquisa se concentra na integração com APIs da seção 6, no armazenamento previsto na seção 11 e nos casos de teste da seção 12. Essa parte conecta comportamento da interface, armazenamento e evidências de funcionamento.

Roteiro explicado

Iniciar separando estado da sessão e persistência. A coleção em memória acompanha a interação; o armazenamento permite recuperá-la depois de recarregar a página. No escopo proposto, os dados ficam no mesmo navegador. Explicar que isso não oferece sincronização entre dispositivos, controle de acesso ou colaboração simultânea.

Organizar leitura e gravação em uma camada própria, para evitar acesso ao armazenamento espalhado pelos componentes. Prever dados ausentes ou inválidos e mensagens de falha. A aplicação deverá continuar abrindo quando houver erro recuperável, sem esconder o problema do usuário.

Explicar a integração futura com uma API: requisições precisam de tratamento de carregamento, erro, resposta vazia e sucesso. Em efeitos, respostas desatualizadas não devem sobrescrever as mais recentes. Também é necessário limpar assinaturas e evitar gravações disparadas apenas pela renderização.

O que deverá entregar e revisar

Camada de persistência e registro dos testes executados. Verificar: envio sem título; cadastro e edição; conclusão e filtros; exclusão confirmada; recuperação após recarregar; persistência inválida; uso por teclado e em tela pequena. Cada caso deve registrar entrada, resultado esperado, resultado obtido e evidência.

Exemplo de validação: cadastrar uma tarefa e conferir o aumento do total; concluí-la e verificar a mudança dos indicadores; aplicar o filtro de pendentes; recarregar e verificar a recuperação da situação salva. Se houver falha, corrigir e repetir o cenário afetado.

Ligação com o próximo integrante

Caio fornece os resultados reais de validação. Luís utiliza essas informações para consolidar limites da aplicação, documentação de execução e condições de publicação.

Síntese da participação

Caio: persistência, tratamento de integração externa e testes com evidências. Os testes só poderão ser descritos como aprovados após sua execução.

5 Quinto integrante

Luís Mário Marques Costa

Matrícula 04214724

Mercado, comparação, documentação e fechamento

Parte atribuída no trabalho

Luís fica responsável por consolidar a análise de mercado e a comparação, além de organizar documentação e publicação. Sua parte reúne as seções 7, 9 e 10, as condições de entrega da seção 12 e as considerações finais da seção 13. A revisão final deve manter ligação entre pesquisa e aplicação.

Roteiro explicado

Iniciar pelas aplicações no mercado e pelos registros de adoção, distinguindo exemplos históricos de arquiteturas atuais. Explicar que pesquisas de desenvolvedores medem uso na amostra: não equivalem a quantidade de vagas, salários ou participação exclusiva de mercado. Evitar conclusões locais sem levantamento específico.

Depois, apresentar React, Angular e Vue segundo os mesmos critérios: aprendizagem, desempenho, escalabilidade, manutenção, popularidade, oportunidades profissionais e adequação. React oferece flexibilidade de composição; Angular reúne uma plataforma integrada; Vue favorece adoção progressiva. São diferenças de proposta, não garantia de superioridade.

Explicar que não foram realizados benchmarks e que as três alternativas podem atender um gerenciador de tarefas. Justificar React pela possibilidade de demonstrar componentes, estado e eventos em escopo acessível ao grupo. Nas considerações finais, relacionar os benefícios às responsabilidades de arquitetura e qualidade.

O que deverá entregar e revisar

README com requisitos de ambiente, versões utilizadas, instalação, execução, testes configurados e build. Os comandos devem ser confirmados no package.json real e testados em ambiente limpo. Organizar o repositório público com código completo e instruções suficientes para outra pessoa reproduzir o funcionamento.

Inserir o link real do GitHub e, quando houver publicação, o endereço real da aplicação. Conferir acesso público, funcionamento dos arquivos e abertura direta de rotas. Não substituir comprovação por links fictícios. Registrar limitações conhecidas e atualizar o relatório conforme a versão entregue.

Fechamento conjunto

Reunir as contribuições dos cinco integrantes e conferir nomes, matrículas, referências, consistência dos conceitos e alinhamento entre relatório, código e aplicação. A revisão final é coletiva, mesmo com Luís organizando o fechamento.

Síntese da participação

Luís: análise de mercado e comparação, consolidação das conclusões, documentação de execução, repositório e organização da publicação.

Sequência de trabalho e conferência final

A ordem abaixo organiza a produção escrita e a execução do projeto. Não corresponde a tempo de fala nem a uma apresentação oral.

1 Mauro

Estabelece problema, objetivos, história e conceitos que sustentam o relatório.

2 Eduardo

Traduz os conceitos em estrutura de componentes e organização visual.

3 Patrick

Define o fluxo de estado, eventos, filtros e indicadores.

4 Caio

Organiza armazenamento, integração externa e comprovação por testes.

5 Luís

Consolida mercado, comparação, conclusão, documentação e entrega técnica.

Antes de entregar

Confirmar as contribuições realizadas; revisar o relatório em conjunto; implementar o escopo; executar e registrar os testes; escrever e verificar as instruções de instalação; disponibilizar código-fonte público; inserir os links reais e revisar as limitações. A divisão não dispensa a leitura do trabalho completo por todos.

Documento de base

Grupo 1. React no desenvolvimento web: fundamentos e análise comparativa com Angular e Vue. Relatório em Word produzido para a disciplina Desenvolvimento Web.

Este roteiro reorganiza o conteúdo desse relatório. As referências técnicas completas permanecem no documento principal. Os cinco participantes devem validar a distribuição antes de utilizá-la como registro definitivo de autoria e contribuição.